DEDEKIND: An Excursion in Structuralist Mereology in C++23

The Dedekind Authors

March 27, 2026

Abstract

Your abstract text.

1 Introduction

A powerful abstraction in mathematical logic is set-builder notation: the ability to define a collection of objects $\{x \in \mathbb{R} \mid \mathbf{P}(x)\}$ based purely on their satisfying a predicate $\mathbf{P}(x)$. In traditional systems programming, however, this declarative elegance is lost. Developers are forced to manually translate these logical intentions into imperative loops, filter functions, and temporary buffers. This translation is not only prone to error but also opaque to the compiler, which cannot optimize a "black-box" loop using the formal properties of the set it is iterating over.

In this paper, we propose dedekind, a C++23 DSL that embeds set-builder notation directly into the host language syntax. By leveraging C++23 Concepts and variadic templates, dedekind allows developers to define complex mathematical species using a syntax that mirrors formal logic. Crucially, these are not mere runtime "filter" objects; they are compile-time structural specifications.

The contributions of this work include:

- The Set-Builder Isomorphism: A syntax-driven mapping that allows expressions like

$$\text{Set}\{\text{x} \,\%\, \mathbb{R} \wedge (\text{x} + 5 < 10)\} \tag{1}$$

  to be resolved at compile-time. We demonstrate how the C++ type system acts as a Constraint Satisfaction Engine to verify the validity of these sets.

- Intensional-to-Extensional Materialization: A strategy for partial, lazy evaluation, where a set defined by a symbolic rule is only converted into machine instructions (extensional data) when an operation requires it. This enables the evaluation of Infinite Sets (e.g., the Naturals $\mathbb{N}$) within a finite execution environment.

- Mereological Pruning of Predicates: Because the library understands the logical structure of the set-builder predicates, the compiler can prune the search space of the set. For example, a set defined by a contradiction $\{x \in \mathbb{S} \mid \mathbf{P}(x) \wedge \neg P(x)\}$ is collapsed into the empty set $\emptyset$ at compile-time, emitting zero machine instructions.

- Zero-Overhead Semantic Mapping: We show that these high-level abstractions result in machine code that is identical to hand-optimized assembly, effectively proving that formal rigor does not necessitate a "performance tax."

By reifying the set-builder notation, dedekind turns the C++ translation unit into a symbolic laboratory. The programmer is free to reason in the language of Cantor and Dedekind, while the compiler produces the performance of a naked C++ loop.

2 Methods

The methodology of `dedekind` is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next.

2.1 The Unified Dedekind Ontology

The `dedekind.ontology` module serves as a central registry of structural truths. It aggregates skeletal, mereological, and algebraic laws to enable the synthesis of the *Continuum* from the *Discrete*. By adopting a *Structuralist* posture—viewing mathematical objects as positions within a structure rather than isolated collections—we facilitate zero-overhead formal verification within the C++ type system.

The ontology is organized into four primary levels of abstraction, mirroring the historical construction of mathematics:

Level -1 to 0: Foundations (Bricks and Cement) At the base lies `:logic`, defining the subobject classifier Ω and predicate logic. This is supported by `:species`, which provides reified machine primitives, and `:category`, which establishes the "Highways" of the system via Morphisms, Functors, and Kleisli Triples.

Level 1: Space (Body and Relation) The `:mereology` partition defines the rules of presence (parts and wholes), while `:order` and `:topology` establish the rules of relation and closeness (posets, lattices, and open sets).

Level 2: Scale (Measurement and Path) The `:sequences` module maps magnitudes to enumerations, providing the necessary machinery for limits and convergence.

Level 3: Soul (Harmony and Action) This level implements the pure algebraic laws (`:algebra`) including Groups, Rings, and Fields, alongside their realized `:morphologies` (Cyclic, Ordered, and Archimedean forms).

Level 4: Realization (The Registry) The final level, `:geometry` and `:numbers`, realizes the continuum through the species registry, formally constructing the inclusion $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$.

2.2 The Semantic Mapping

Crucially, the `dedekind.ontology` acts as the definitive mapping between formal mathematical nomenclature and the concrete syntax of the C++23 language. Rather than introducing a foreign vocabulary, we anchor established axioms directly into the standard operator set and primitive types. For instance, the mereological *Sum* and *Product* are reified as the bitwise operators `|` and `&`, while the *Mereological Complement* (the remainder) is bound to the negation operator `!`.

By establishing this explicit correspondence—where `operator<=` serves as the formal proof of *Parthood* ($\sqsubseteq$) and `operator/` represents the *Quotienting* of a species—we ensure that the resulting code is not merely a simulation of a mathematical model, but a direct execution of it. This semantic alignment allows the programmer to write expressions that are readable as equations, while the compiler treats them as a sequence of verifiable type-transformations. In this architecture, the C++ `concept` becomes the "Gatekeeper of Truth," ensuring that a species cannot participate in an operation (such as a Join-Semilattice union) unless it has first provided a manual or structural certification of its algebraic invariants.

Table 1: The Dedekind Semantic Mapping: A Registry of Formal Correspondences

Mathematical Concept	**Symbol**	**C++ Operator / Type**	**C++ Concept**	**Basis**
Category Theory				
Kleisli Extension	$(T, \eta, \gg=)$	`operator»=`	`IsKleisliExtension`	`IsAssociative`
Kleisli Extension	$\gg=$	`operator»=`	`IsKleisliExtension`	`IsAssociative`
Kleisli Composition	$\circ\_K$	`operator»`	`IsMonad`	`IsAssociative`
Co-Kleisli Extension	$\gg=_{co}$	`operator«=`	`IsComonad`	`IsAssociative`
Morphic Injection (Unit)	η	`into<F>` / `singleton`	`IsMonad`	$\eta : T \to F(T)$
Morphic Extraction (Counit)	ε	`extract<F>`	`IsComonad`	$\varepsilon : F(T) \to T$
Characteristic Morphism	χ	`operator()`	`IsCharacteristic`	
Co-Kleisli Composition	$\circ\_CoK$	`operator«`	`IsComonad`	
Counit (Extraction)	ε	`extract<F>`	`IsComonad`	
Mereology				
Proper Parthood	χ	`operator()`	`IsProperPart`	`IsCharacteristic`
Parthood (Pre-order)	$\sqsubseteq$	`operator<=`	`IsPartOf`	
Mereological Sum (Join)	$\sqcup$	`operator|`	`IsJoinSemilattice`	
Mereological Product (Meet)	$\sqcap$	`operator&`	`IsMeetSemilattice`	
Universal Complement	$\neg$	`operator!`	`IsComplemented`	
Initial Object (Empty)	$\emptyset$	`∅<T>`	`IsInitialObject`	
Terminal Object (Universal)	Ω	`Ω<T>`	`IsTerminalObject`	
Set Theory				
Membership	$\in$	`operator()`	`IsSet`	`IsProperPart`
Subset	$\subseteq$	`operator<=`	`IsSubset`	`IsPartOf`
Countable Infinity	$\aleph_0$	`ℵ_0`	`IsCardinality`	
Continuum Cardinality	$\beth_1$	`ℶ_1`	`IsCardinality`	
Order Theory				
Pre-order	$\sqsubseteq$	`operator<=`	`IsPartiallyOrdered`	`IsPartOf`
Algebra				
Structural Origin (Zero)	0	`T::origin()`	`IsPointed`	
Additive Composition	$+$	`operator+`	`IsGroup`	
Additive Inverse	$-$	`operator-`	`IsGroup`	
Multiplicative Composition	$\times$	`operator*`	`IsRing`	
Multiplicative Inverse	$/, {}^{-1}$	`operator/`	`IsField`	
Associativity	$(a \circ b) \circ c$	`is_associative_v`	`IsSemigroupoid`	
Idempotency	$a \circ a = a$	`is_idempotent_v`	`IsIdempotent`	
Magnitude / Count	$\|S\|$	`s.size()` / `s.cardinality()`	`IsCardinality`	
Supremum / Infimum	$\sup, \inf$	`s.supremum(), s.infimum()`	`HasExtrema`	

2.3 The Registry of Structural Proofs

The transition from a raw computational graph to a simplified mathematical identity requires a formal "Seal of Truth." In `dedekind`, we implement a *Traits Registry* that forces the developer to explicitly certify the equational axioms of a species before it can participate in the lattice-based optimizations of the dispatcher.

This is achieved through a dual-layered system of **Trait Ledgers** and **Discovery Concepts**. For each fundamental algebraic law, we define a primary template (the ledger) which defaults to `false`:

```
export template <typename T, typename Op>
struct is_associative : std::false_type {};

export template <typename T, typename Op>
inline constexpr bool is_associative_v = is_associative<T, Op>::value;
```

Listing 1: The Associativity Ledger in :species

A species S is only recognized as a `IsSemigroupoid` if it (or the ontology) provides a specialization of this ledger. To allow for "Interoperable Discovery," we provide a fallback mechanism that probes the species for internal certification:

```
export template <typename T, typename Op>
concept IsAssociative = IsMagmoid<T, Op> && requires {
    requires is_associative_v<T, Op>;
};

// Discovery Fallback: Probing the Species for an Intrinsic Proof
template <typename T, typename Op>
  requires requires { T::is_associative_for<Op>; }
struct is_associative<T, Op> : T::template is_associative_for<Op> {};
```

Listing 2: The Concept of Formal Association

By anchoring concepts like `IsJoinSemilattice` in these requirements (requiring both `IsAssociative` and `IsIdempotent`), we ensure that the *Lattice Dispatcher* never performs a "speculative" pruning. If the developer has not provided the proof, the dispatcher falls back to a safe, unoptimized `UnionSet` synthesis. This "Axiom of Responsibility" ensures that the performance of the "Naked Loop" is always mathematically justified by a prior formal certification.

2.4 Reverse Curry-Howard Validation

To verify the integrity of the proposed mapping between formal axioms and C++ concepts, we employ a "Reverse Curry-Howard" validation strategy. Rather than simply assuming the correctness of our ontological translations, we use the inherent properties of C++ built-in types as an empirical proof of the reverse-engineered axioms.

By executing a suite of `static_assert` evaluations on primitive types—such as verifying that `bool` satisfies the requirements of an `IsJoinSemilattice` or that bitwise operations on `uint32_t` adhere to the `IsMereologicalLattice` consistency axioms—we confirm that the machine-level behavior is isomorphic to the formal mathematical theory. In this posture, the built-in types serve as the "Initial Models" for our concepts. If the compiler successfully verifies that a standard integer under bitwise conjunction satisfies our `IsMeetSemilattice` proof, it provides an automated, formal confirmation that our C++ concepts correctly capture the essential structural invariants found in the mathematical literature.

3 Implementation: The Functor Highway

The operational efficiency of `dedekind` is achieved through a technique we term *Synthetic Inference*. This level represents the "Skeletal Foundation" of the ontology, where machine-level instructions are unified with the abstract laws of Category Theory. By treating the C++ type system as a formal proof assistant, we ensure structural integrity via compile-time verification, or *Static Mereology*.

3.1 The Action-First Bootstrapping Strategy

In textbook Category Theory, a Monad is traditionally defined as a *Monoid in the category of Endofunctors*. However, the structural implementation of this definition in C++ faces significant language constraints:

1. **Partial Specialization Limitations**: C++ forbids the partial specialization of function templates (e.g., `fmap<F>`), preventing a centralized, generic definition for disparate species.
2. **ADL Resolution**: Function templates called via explicit name-lookup cannot trigger Argument-Dependent Lookup (ADL), making it difficult for the ontology to "discover" mappings for types defined in downstream modules.

To resolve this circular dependency, `dedekind` implements an *Action-First* bootstrapping strategy. By defining Monads and Comonads as *Extension Systems* (Kleisli Triples consisting of η/ε and $\gg= / \ll=$), we utilize operator overloading on concrete objects. This triggers ADL at the call site, allowing the Level 0 foundation to instantiate a derived `fmap` without prior knowledge of Level 1 species.

The Precedence-Associativity Constraint A significant friction point in embedding categorical notation within the C++ grammar is the language's fixed operator precedence and associativity table. While the *Highway Notation* utilizes `operator»=` to model the Monadic Bind ($\gg=$), the ISO C++ standard defines compound assignment operators (including »=, «=, +=, *=, etc.) with **right-to-left associativity**. Consequently, a standard monadic chain such as $m \gg= f \gg= g$—which mathematically implies a left-to-right flow of data—is parsed by the compiler as $m \gg= (f \gg= g)$.

Because the right-hand operands (f and g) are typically Kleisli arrows (lambdas) rather than monadic types, this default parsing triggers a candidate resolution failure, as the compiler attempts to "bind" two functions before involving the monadic context m. To maintain the "Forward Highway" semantics without introducing a non-standard pre-processor, `dedekind` currently requires explicit grouping, e.g., `(m »= f) »= g`. This constraint highlights a fundamental tension between the declarative goals of the DSL and the imperative heritage of the host language's grammar, necessitating a "parenthetical tax" to enforce the correct directional vector of the computation.

3.2 The Unified Highway Bridge

The `fmap` dispatcher serves as the implementation of this strategy. It automates the derivation of functorial mappings from the underlying monadic "Push" or comonadic "Pull."

```
export template <template <typename...> typename F, typename Arrow,
                 typename T = typename Arrow::Domain,
                 typename U = typename Arrow::Codomain>
  requires IsArrow<Arrow, T, U>
constexpr IsArrow<F<T>, F<U>> auto fmap(Arrow f) {
  if constexpr (std::same_as<F<T>, T>) {
```

```
    return f; // Identity Functor
  } else if constexpr (IsKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = m >>= (eta o f) */
    return arrow<F<T>, F<U>>([f](const F<T>& m) {
      return m >>= [f](const T& x) { return into<F, U>(f(x)); };
    });
  } else if constexpr (IsCoKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = w <<= (f o epsilon) */
    return arrow<F<T>, F<U>>([f](const F<T>& w) {
      return w <<= [f](const F<T>& ctx) { return f(extract<F, T>(ctx)); };
    });
  } else {
    static_assert(sizeof(T) == 0, "Ontology Error: Species lacks highway structure."
  }
}
```

Listing 3: The Unified Highway Bridge (fmap derivation)

3.3 Highway Notation: Directional Vectors

To facilitate a readable "Algebra of Programming," we map categorical data flows to directional operators, creating a "Highway Notation" that reflects the vector of computation across the ontology:

- `value » into<F>` : Represents η (Unit), lifting a species into a context.
- `box « extract<F>` : Represents ε (Counit), sampling a species from a context.
- `box »= f` : Represents *Bind*, the forward monadic composition (Left-to-Right).
- `box «= f` : Represents *Extend*, the contextual comonadic composition (Right-to-Left).

```
// Convert plain lambdas into tagged endomorphisms:
auto f = endo<int>([](int x) { return x + 1; });
auto g = endo<int>([](int x) { return x * 2; });

// Morphism composition: (h = g . f):
auto h = f >> g;

// Lifting into the Box and applying the composed morphism:
auto result = 5 >> into<Box> >> fmap<Box>(h);

CHECK(result == Box<int>{12});
```

Listing 4: Example of fused Kleisli "Highway" composition via the functor law.

```
// Define two monadic arrows (Species -> Boxed Species)
auto f = [](int x) { return pure(x + 1); };
auto g = [](int x) { return pure(x * 2); };

// Chain composition via Bind (>>=)
// Starting with a raw value, lifting it, then chaining the monadic arrows.
auto x = (5 >> into<Box> >>= f) >>= g;

// (5 + 1) * 2 = 12, wrapped in a Box
CHECK(x == Box<int>{12});

// Verification of the Monadic Associativity Law:
```

```
// (m >>= f) >>= g  is equivalent to  m >>= (x => f(x) >>= g)
auto lh = (5 >> into<Box> >>= f) >>= g;

auto rh = 5 >> into<Box> >>= [&](int x) { return f(x) >>= g; };

CHECK(lh == rh);
STATIC_CHECK(std::same_as<decltype(x), Box<int>>);
```

Listing 5: Chain Composition (Bind / »=)

This mapping ensures that the "Categorical Cement" holding the "Algebraic Bricks" together is both syntactically expressive and computationally invisible.